

Gestione Utenti e Permessi

Modulo: 2A | **Corso:** Lab Amministrazione di Sistemi T

Prerequisiti: Moduli 0A-1B completati

VM: `cd ~/sysAdmin-lab && vagrant up --provider=virtualbox && vagrant ssh`

Il modello utenti di Linux

Linux non conosce i nomi: internamente ogni utente è un numero. `vagrant`, `root`, `alice` sono tutte etichette leggibili che il sistema traduce consultando `/etc/passwd`. Il comando che mostra i propri numeri è `id`:

```
id
# uid=1000(vagrant) gid=1000(vagrant) groups=1000(vagrant)
```

Risposta alla domanda — differenza tra `gid` e `groups`

`gid` è il **gruppo primario**: il gruppo assegnato di default a ogni file creato dall'utente. È uno solo. `groups` è la lista di **tutti i gruppi** a cui l'utente appartiene — il primario più tutti i gruppi secondari aggiunti con `usermod -aG`. Nel caso di `vagrant` appena installato, i due coincidono perché non è stato aggiunto a nessun gruppo secondario. Se aggiungiamo `vagrant` al gruppo `sudo`, l'output diventerebbe `groups=1000(vagrant),27(sudo)`.

La convenzione degli UID

Intervallo	Destinazione
0	Root — sempre e solo
1 – 999	Utenti di sistema (demoni, servizi: <code>www-data</code> , <code>sshd</code> , <code>nobody</code> ...)
1000+	Utenti fisici reali

```
# Vedere tutti gli utenti del sistema
cat /etc/passwd
```

```
# Vedere solo gli utenti fisici (UID ≥ 1000)
awk -F: '$3 >= 1000 {print $1, $3}' /etc/passwd
```

Risposta alla domanda — posso vedere solo la lista degli utenti fisici?

Sì, il comando `awk -F: '$3 >= 1000 {print $1, $3}' /etc/passwd` filtra le righe di `/etc/passwd` mostrando solo gli utenti con `UID ≥ 1000`, che per convenzione sono quelli fisici. `awk -F:` imposta : come separatore di campo; `$3` è il terzo campo (l'UID); `$1` è il primo (il nome utente).

Struttura di /etc/passwd

```
vagrant : x : 1000 : 1000 : , , , : /home/vagrant : /bin/bash
|       |       |       |       |       |
nome  pw(x)  UID   GID   GECOS      home      shell
```

- **pw:** sempre x — indica che la password è in `/etc/shadow`, non qui
- **GECOS:** campo informativo opzionale (nome completo, telefono, ecc.) — spesso lasciato vuoto (, , ,)
- **shell:** interprete assegnato al login; `/bin/bash` per utenti normali, `/usr/sbin/nologin` o `/bin/false` per utenti di sistema che non devono effettuare login interattivi

Storage delle password — /etc/shadow

Le password non sono in `/etc/passwd` (leggibile da tutti) ma in `/etc/shadow`, accessibile solo da root. Contiene gli hash delle password e le politiche di scadenza.

```
sudo cat /etc/shadow | grep vagrant
```

Struttura di una riga:

```
vagrant : $6$abc123... : 19800 : 0 : 99999 : 7 : : :
|       |       |       |       |       |
nome    hash pw   gg  ult  min    max  warn
```

Campo	Significato
nome	Nome utente
hash pw	Hash della password (\$6\$ = algoritmo SHA-512)
gg ult	Giorni dall'epoch (1 gen 1970) dell'ultimo cambio password
min	Giorni minimi prima di poter cambiare password di nuovo (0 = nessun limite)
max	Giorni massimi prima che la password scada (99999 ≈ mai)
warn	Giorni di preavviso prima della scadenza

Il fatto che `/etc/shadow` sia leggibile solo da root è una misura di sicurezza critica: se fosse leggibile da tutti, un attaccante potrebbe copiare gli hash e craccarne le password offline con strumenti come `john` o `hashcat`.

Gestione degli utenti

Creare un utente — `useradd`

```
sudo useradd -m -s /bin/bash alice
sudo passwd alice
```

Flag	Significato
<code>-m</code>	Crea la home directory <code>/home/alice</code>
<code>-s /bin/bash</code>	Assegna Bash come shell di login

Senza `-m` la home non viene creata. Senza `-s` viene usata la shell di default del sistema (spesso `sh`).

Modificare un utente — `usermod`

`usermod` permette di modificare quasi ogni attributo di un utente esistente:

Comando	Effetto
<code>sudo usermod -aG gruppo nome</code>	Aggiunge l'utente a un gruppo secondario (senza rimuoverlo dagli altri)
<code>sudo usermod -s /bin/zsh nome</code>	Cambia la shell di login
<code>sudo usermod -d /nuova/home nome</code>	Cambia la home directory
<code>sudo usermod -l nuovo_nome vecchio_nome</code>	Rinomina l'utente
<code>sudo usermod -L nome</code>	Blocca l'account (lock)
<code>sudo usermod -U nome</code>	Sblocca l'account (unlock)

Il flag `-aG` è la combinazione più usata: `-a` significa **append** (aggiungi senza sostituire), `-G` specifica il gruppo. Usare `-G` senza `-a` sostituirebbe tutti i gruppi secondari con quello indicato — errore comune e potenzialmente distruttivo.

Cambiare utente — `su`

```
su - alice    # con trattino: carica l'ambiente completo di alice
su alice      # senza trattino: cambia utente ma mantiene l'ambiente corrente
```

Il trattino in su - alice è importante: carica le variabili d'ambiente di alice (\$HOME, \$PATH, ecc.) e posiziona nella sua home. Senza trattino ci si ritrova nell'ambiente del chiamante — comportamento quasi sempre indesiderato, fonte di bug difficili da diagnosticare.

```
exit    # per tornare all'utente precedente
```

Eliminare un utente — userdel

```
sudo userdel alice      # elimina l'utente, lascia la home intatta
sudo userdel -r alice   # elimina l'utente E la home directory
```

Proprietà di un file — chown e chgrp

Alla creazione, ogni file viene associato all'utente che lo ha creato (owner) e al suo gruppo primario. Questi attributi si modificano con chown e chgrp.

```
echo "file di vagrant" > file_vagrant.txt
ls -la file_vagrant.txt
# output: -rw-r--r-- 1 vagrant vagrant ...
```

```
# Cambiare solo il proprietario
sudo chown alice file_vagrant.txt
# output: -rw-r--r-- 1 alice vagrant ...
```

```
# Cambiare proprietario e gruppo insieme
sudo chown alice:alice file_vagrant.txt
# output: -rw-r--r-- 1 alice alice ...
```

```
# Cambiare solo il gruppo
sudo chgrp vagrant file_vagrant.txt
# output: -rw-r--r-- 1 alice vagrant ...
```

chown richiede sudo perché solo root può trasferire la proprietà di un file. Un utente può modificare il gruppo solo verso un gruppo di cui fa già parte.

Permessi — chmod

Ogni file ha una stringa di 10 caratteri che ne descrive i permessi:

```
- r w x   r - x   r - -
| | |     | | |   | | |
| owner   group   others
```

|
tipo: - = file, d = directory, l = link simbolico

Le tre lettere rwx valgono: - r = 4 (read) - w = 2 (write) - x = 1 (execute)

La somma dei tre valori per ciascun blocco produce la notazione ottale:

Stringa	Ottale	Uso tipico
rw-r--r--	644	File normale — owner scrive, tutti leggono
rxwxr-xr-x	755	Script/directory — owner tutto, altri leggono+eseguono
rw-----	600	File privato — solo owner
rxwx-----	700	Directory/script privato — solo owner
rxwxrwx---	770	Condiviso tra owner e gruppo, nessun altro
rxwxrwxrwx	777	Tutti fanno tutto — pericoloso

Notazione ottale (la più usata)

```
chmod 644 file.txt
chmod 755 script.sh
chmod 700 file_privato.txt
chmod 000 file_test.txt    # nessuno può fare nulla
```

Verifica che il blocco funzioni davvero

```
echo "segreto" > file_test.txt
chmod 000 file_test.txt
cat file_test.txt          # → Permission denied
chmod 644 file_test.txt
cat file_test.txt          # → segreto
```

Notazione simbolica (utile per modifiche puntuali)

```
chmod u+x script.sh      # aggiunge esecuzione all'owner
chmod g-w file.txt        # rimuove scrittura al gruppo
chmod o-r file.txt        # rimuove lettura agli altri
chmod a+r file.txt        # aggiunge lettura a tutti
```

u = user (owner), g = group, o = others, a = all.

Come Linux decide i permessi — composizione

Quando un utente A accede a un file, il kernel segue questo schema **deterministico**:

A è il proprietario (U)?

Sì → applica i permessi di U. Fine.

No → A appartiene al gruppo proprietario (G)?

Sì → applica i permessi di G. Fine.

No → applica i permessi di O (others). Fine.

Punto controintuitivo: i blocchi si escludono a vicenda. Se A è il proprietario del file, si applicano **solo** i permessi di U — anche se i permessi di G o O fossero più permissivi. Esempio:

```
# File con questi permessi: ---rwxrwx (000 per owner, 777 per group e others)
# Se sei il proprietario: non puoi fare nulla
# Se sei nel gruppo: puoi fare tutto
```

Questo è il motivo per cui a volte un proprietario ha meno accesso di un altro utente sullo stesso file — è by design, non un bug.

umask — permessi di default alla creazione

Quando crei un file o una directory, Linux non assegna permessi fissi: parte da un massimo e **sottrae** la umask.

Tipo	Massimo di partenza	Umask tipica (002)	Risultato
File	666 (rw-rw-rw-)	002	664 (rw-rw-r-)
Directory	777 (rwxrwxrwx)	002	775 (rwxrwxr-x)

Il massimo per i file è 666 (non 777) perché l'eseguibilità è un'eccezione — non si concede per default.

```
# Vedere la umask corrente
umask
# → 0002
```

```
# Cambiarla per la sessione corrente
umask 0027 # owner tutto, gruppo legge, others nulla
```

```
# Verificare l'effetto
touch file_test && ls -la file_test
# → -rw-r----- (640)
```

La umask è per sessione: per renderla persistente va messa in ~/.bashrc.

Umask usata nel lab: il prof usa umask 0002 (rimuove write agli others), che va inserita nel .bashrc degli utenti collaborativi. In questo modo i file creati nella directory condivisa sono leggibili e scrivibili da tutto il gruppo, ma non da estranei.

Bit speciali — SUID, SGID, Sticky

Oltre ai 9 bit rwx, esistono 3 bit speciali che alterano il comportamento del sistema.

SUID (bit 11) — Set User ID

Applicato a un **file eseguibile**: quando viene lanciato, il processo gira con l'identità del **proprietario del file**, non di chi lo esegue.

```
# Esempio classico: passwd
ls -la /usr/bin/passwd
# → -rwsr-xr-x 1 root root ...
#           ^
#           's' al posto di 'x' = SUID attivo
```

passwd deve scrivere in /etc/shadow (accessibile solo a root) — lo fa grazie al SUID: chiunque lo esegua, il processo ottiene i privilegi di root. Questo è il meccanismo che permette agli utenti di cambiare la propria password.

```
chmod 4755 mio_eseguibile # imposta SUID + rwxr-xr-x
chmod u+s mio_eseguibile # equivalente simbolico
```

SGID (bit 10) — Set Group ID

Su file eseguibili: funziona come SUID ma per il gruppo — il processo gira con il gruppo proprietario del file.

Su directory: questo è il caso più utile in pratica. Se una directory ha SGID impostato, i file creati al suo interno ereditano il **gruppo della directory** (non il gruppo primario del creatore).

```
# Senza SGID: alice crea un file → gruppo = alice (suo gruppo primario)
# Con SGID:   alice crea un file → gruppo = programmatori (gruppo della dir)
```

```
chmod 2770 /home/programmatori # SGID + rwxrwx---
chmod g+s /home/programmatori # equivalente simbolico
```

Questo è esattamente il meccanismo del lab: con umask 0002 + SGID sulla directory, qualsiasi file creato da maria o piero risulta automaticamente scrivibile da entrambi, senza dover fare chgrp manualmente ogni volta.

Sticky bit (bit 9)

Applicato a una **directory**: i file al suo interno possono essere cancellati solo dal loro proprietario, anche se la directory è world-writable.

```
ls -la /tmp
# → drwxrwxrwt (nota la 't' finale = sticky)
```

/tmp è scrivibile da tutti, ma nessun utente può cancellare i file degli altri — solo i rispettivi proprietari (o root).

```
chmod 1777 /tmp      # sticky + rwxrwxrwx
chmod +t    /tmp      # equivalente simbolico
```

Lettura nella stringa permessi

```
- r w s  r - s  r - t
      ^      ^      ^
      SUID   SGID   Sticky
      (su owner) (su grp) (su dir)
```

La lettera s compare al posto della x del blocco corrispondente. Se il bit eseguibile non è impostato, appare S maiuscola (bit impostato ma senza effetto).

Notazione ottale con bit speciali

Il quarto cifra ottale (quella davanti) codifica i tre bit speciali:

Valore	Bit impostato
4xxx	SUID
2xxx	SGID
1xxx	Sticky
6xxx	SUID + SGID

```
chmod 2770 /home/programmatori # SGID + rwxrwx---
chmod 4755 /usr/bin/mioprogram # SUID + rwxr-xr-x
chmod 1777 /tmp                 # Sticky + rwxrwxrwx
```

Privilegi elevati — sudo

sudo (superuser do) permette di eseguire un singolo comando come root senza diventare root permanentemente. La configurazione che stabilisce chi può fare cosa sta in /etc/sudoers.

```
# Non modificare mai /etc/sudoers direttamente – usare visudo
sudo cat /etc/sudoers
```

La riga chiave per Debian è:

```
%sudo    ALL=(ALL:ALL) ALL
```

Traduzione: il gruppo sudo (%sudo) può eseguire qualsiasi comando su qualsiasi host, come qualsiasi utente.

sudo -l — interpretare l’output

```
sudo -l
```

Output sulla VM Vagrant:

Matching Defaults entries for vagrant on bookworm:

```
env_reset, mail_badpass,
secure_path=...,
use_pty
```

User vagrant may run the following commands on bookworm:

```
(ALL) NOPASSWD: ALL
```

Risposta alla domanda — cosa ottengo materialmente da sudo -l?

sudo -l elenca tutti i comandi che l’utente corrente può eseguire con sudo, in che contesto, e se è richiesta o meno la password. È uno dei primi comandi eseguiti da un attaccante dopo aver ottenuto accesso a una macchina — per questo è fondamentale saperlo leggere.

Per interpretare l’output bisogna conoscere la struttura completa di una regola sudoers:

```
CHI      DOVE  = (COME_CHI)  COSA
vagrant ALL   =      (ALL)    NOPASSWD: ALL
```

- **CHI** (vagrant) — l’utente a cui si applica la regola. Vale solo per vagrant.
- **DOVE** (ALL) — su quale macchina vale. In infrastrutture con /etc/sudoers condiviso via rete si può limitare a un host specifico; ALL significa qualsiasi.
- **COME CHI** (ALL tra parentesi) — quale utente vagrant può **impersonare** quando usa sudo. Per default sudo esegue come root,

ma `sudo -u alice` comando permette di eseguire come alice. Il valore `ALL` significa che vagrant può impersonare qualsiasi utente del sistema, non solo root. Questo campo non amplia a chi si applica la regola — si applica sempre e solo a vagrant — ma allarga la gamma di identità che vagrant può assumere.

- **COSA** (ALL finale) — quali comandi sono permessi.
- **NOPASSWD** — nessuna password richiesta per nessuno di questi comandi.

La configurazione complessiva è la più permissiva possibile: normale in un ambiente di laboratorio Vagrant, pericolosa in produzione.

Implicazione pratica del campo COME CHI — accesso a risorse di altri utenti

Il campo (ALL) non riguarda solo root. Significa che vagrant può impersonare **qualsiasi** utente del sistema tramite `sudo -u nome comando`. In pratica: se alice appartiene al gruppo progetto e vagrant no, vagrant è normalmente escluso dalla directory `/srv/progetto` (permessi 770). Ma con (ALL) in sudoers può aggirare questo limite:

```
# Come vagrant, accesso diretto → negato
```

```
cd /srv/progetto
```

```
# -bash: cd: /srv/progetto/: Permission denied
```

```
# Come vagrant, tramite sudo -u alice → funziona
```

```
sudo -u alice cat /srv/progetto/alice.txt
```

```
# → file di alice
```

Vagrant non entra nella directory — esegue `cat` con l'identità di alice, e il filesystem vede alice (membro del gruppo progetto), non vagrant. Il risultato è accesso completo a qualsiasi risorsa legata all'identità di qualsiasi utente del sistema: file privati, chiavi SSH, dati applicativi.

La contromisura è restringere il campo COME CHI nella regola sudoers. Se vagrant ha bisogno di `sudo` solo per operazioni di sistema come root, la regola corretta è:

```
vagrant ALL=(root) NOPASSWD: ALL
```

Con (root) al posto di (ALL), `sudo -u alice` viene rifiutato — vagrant può eseguire solo come root, non come utenti arbitrari.

Risposta alla domanda — perché vagrant ha sudo se groups vagrant non mostra il gruppo sudo?

Vagrant configura i privilegi di root tramite un file dedicato in `/etc/sudoers.d/vagrant`, non tramite l'appartenenza al gruppo

sudo. I file in /etc/sudoers.d/ vengono letti da sudo esattamente come /etc/sudoers — sono solo file separati per organizzare meglio le regole. Puoi verificarlo:

```
sudo cat /etc/sudoers.d/vagrant
# output: vagrant ALL=(ALL) NOPASSWD: ALL
```

Quindi vagrant ha accesso root completo senza password, indipendentemente dai suoi gruppi. Il gruppo sudo è il meccanismo standard di Debian/Ubuntu; /etc/sudoers.d/ è il meccanismo che Vagrant usa per configurare la VM senza toccare il file principale.

Esercizio integrato — directory condivisa

```
# Creare gruppo e utenti
sudo groupadd progetto
sudo useradd -m -s /bin/bash alice
sudo useradd -m -s /bin/bash bob
sudo passwd alice # imposta password
sudo passwd bob

# Aggiungere alice e bob al gruppo progetto
sudo usermod -aG progetto alice
sudo usermod -aG progetto bob
groups alice # alice : alice progetto
groups bob # bob : bob progetto

# Creare la directory condivisa
sudo mkdir /srv/progetto
sudo chown root:progetto /srv/progetto
sudo chmod 770 /srv/progetto
ls -la /srv/
```

Risposta alla domanda — perché cambiamo il proprietario della directory con `chown root:progetto`?

Il comando non cambia il proprietario individuale (che resta root) ma il **gruppo proprietario**: lo imposta a progetto. Questo è il meccanismo che rende la directory condivisa tra alice e bob senza che uno sia il proprietario dell'altro. Con `chmod 770`, il blocco dei permessi per il gruppo è `rwX` — quindi tutti i membri del gruppo progetto possono creare, leggere e modificare file dentro quella directory. Il proprietario individuale root non è significativo qui; quello che conta è che il gruppo sia progetto.

Risposta alla domanda — /srv è una cartella preesistente? cosa sono . e ..?

Sì, /srv è una directory standard del filesystem Linux (definita dallo standard FHS — Filesystem Hierarchy Standard). È destinata ai dati dei servizi ospitati dalla macchina (es. file di un server web, file FTP, ecc.). Sulla VM Debian appena installata è vuota — l'unico contenuto è la directory progetto che hai creato tu.

Le voci . e .. che appaiono in ogni `ls -la` **non sono file**: sono voci speciali del filesystem che ogni directory contiene sempre. . è un riferimento alla directory stessa; .. è un riferimento alla directory genitore. Compaiono in qualsiasi `ls -la` su qualsiasi directory Linux — sono struttura del filesystem, non file creati da qualcuno.

```
# Test come alice
su - alice
cd /srv/progetto
echo "file di alice" > alice.txt
ls -la
exit
```

```
# Test come bob
su - bob
cd /srv/progetto
cat alice.txt           # funziona – bob è nel gruppo progetto
exit
```

```
# Test come utente fuori dal gruppo
sudo useradd -m -s /bin/bash sconosciuto
sudo passwd sconosciuto
su - sconosciuto
cd /srv/progetto       # → Permission denied (sconosciuto non è nel gruppo)
exit
```

Il risultato dimostra concretamente il funzionamento del modello a tre categorie di Linux: owner, group, others. sconosciuto cade nel blocco others, che per questa directory ha permessi ---.

Connessione con Security

Questo modulo è il prerequisito concettuale diretto per **privilege escalation**:

- Dopo aver ottenuto accesso come utente non privilegiato, l'attaccante esegue subito `sudo -l`: se trova NOPASSWD: ALL o un singolo interprete

(python, vim, bash) eseguibile con sudo, può ottenere una shell root in secondi

- File con permessi 777, o file di configurazione critici scrivibili da utenti normali, sono vettori di escalation classici
- /etc/shadow leggibile da non-root è una vulnerabilità critica: gli hash si craccano offline
- Utenti nel gruppo sudo senza necessità operativa reale ampliano inutilmente la superficie d'attacco
- La directory condivisa con 770 è la configurazione corretta: un 777 sulla stessa directory permetterebbe a qualsiasi utente del sistema di accedere ai file

Riepilogo comandi

Comando	Funzione
id [nome]	Mostra UID, GID e tutti i gruppi
groups [nome]	Mostra i gruppi dell'utente
cat /etc/passwd	Lista tutti gli utenti del sistema
awk -F: '\$3>=1000 {print \$1}' /etc/passwd	Lista solo gli utenti fisici
sudo cat /etc/shadow	Mostra hash password (solo root)
sudo useradd -m -s /bin/bash nome	Crea utente con home e shell Bash
sudo passwd nome	Imposta/cambia password utente
sudo usermod -aG gruppo nome	Aggiunge utente a un gruppo
sudo usermod -s /bin/zsh nome	Cambia la shell
sudo userdel -r nome	Elimina utente e home directory
sudo groupadd nome	Crea un nuovo gruppo
su - nome	Cambia utente con caricamento ambiente completo
sudo comando	Esegue comando come root
sudo -i	Apri shell root completa
sudo -l	Elenca comandi eseguibili con sudo (e se serve password)
sudo cat /etc/sudoers.d/vagrant	Mostra le regole sudo specifiche di Vagrant
chmod 755 file	Modifica permessi in notazione ottale
chmod u+x file	Modifica permessi in notazione simbolica
chmod 2770 dir	Imposta SGID + rwxrwx— su una directory
chmod g+s dir	Imposta SGID in notazione simbolica
chmod +t dir	Imposta sticky bit

Comando	Funzione
<code>sudo chown utente:gruppo file</code>	Cambia proprietario e gruppo
<code>sudo chgrp gruppo file</code>	Cambia solo il gruppo
<code>umask</code>	Mostra la umask corrente
<code>umask 0002</code>	Imposta umask per la sessione